Neo BookFinder

Description

Book Finder is a frontend-only web application that lets users search for books by title, author, or keyword and browse the results — covers, authors, publish years — pulled live from the free, key-less Open Library API. Selecting a book opens a detail view with a fuller description, subjects/genres, and publish information. The motivation is to give students a beginner-friendly, low-setup project that still covers the core skills used across almost any frontend or API-driven work: making HTTP requests, handling asynchronous JSON responses, and rendering dynamic UI — without needing a backend, database, or API key.

The project is scoped to plain HTML, CSS, and JavaScript so it can be built and run entirely in the browser with no server setup. It is intentionally simple to start (a single search box and a results grid) but leaves clear, natural directions to extend for students who want to go further.

Requirements

Programming languages

- HTML, CSS, and vanilla JavaScript — no framework required (though it may optionally be rebuilt in React later as a stretch exercise).
- No backend and no database — all data comes directly from the Open Library REST API via fetch().

Component Construction

- SearchBar — input field and button that captures the user's search term.
- ResultsGrid — renders the list of matching books as cards after a search.
- BookCard — displays a single book's cover, title, author, and first publish year.
- DetailView — shows expanded information for a selected book (description, subjects, publish info).

Feature

1. **Search**
    - Search books by title, author, or general keyword using the Open Library Search API.
    - Display results as a responsive grid of cards, each showing cover image, title, author, and year.

- Handle the empty-input case and the no-results-found case with clear on-screen messaging.

2. **Book Details**
   - Clicking a result opens a detail view with a longer description, subjects/genres, and edition information.
   - Gracefully handle missing data — not every book has a cover, description, or subject list.
3. **UI/UX States**
   - Loading indicator while a search request is in flight.
   - Friendly error message if the API request fails (e.g., network issue).
   - Responsive layout that works on both desktop and mobile widths.
4. **Stretch Features (optional)**
   - Save favorite books to a personal list using localStorage.
   - Filter or sort results by publish year or subject.
   - "Load more" / pagination for longer result sets.

Supporting Material

- Open Library Search API documentation [https://openlibrary.org/dev/docs/api/search]
- Open Library Covers API [https://openlibrary.org/dev/docs/api/covers]
- MDN — Using the Fetch API [https://developer.mozilla.org/en-US/docs/Web/API/Fetch_API/Using_Fetch]

Deliverables

- A working web app (HTML/CSS/JS) deployed or runnable locally, demonstrating search and detail views.
- A short screen recording or GIF showing a search, browsing results, and viewing book details.
- Screenshots of the loading, empty-result, and error states.

Code Repository and Documentation

- All source code related to the project must be pushed to a GitHub repository.
  - Clear, organized file structure (e.g., separate files or folders for markup, styles, and script logic).

- Meaningful commit messages reflecting incremental progress (search → results grid → detail view → polish).
- A demonstration video or GIF of the working app.
- A README.md file must be included in the repository, clearly explaining:
  - The purpose of the project
  - Setup instructions (e.g., how to open/run the app locally)
  - Which Open Library endpoints are used
  - Known limitations (e.g., missing covers/descriptions for some books)
  - Any extra/stretch features implemented

Final Evaluation

Final assessment will be based on:

- Clearest, most accurate technical explanation of the root cause.
- Most useful and accurate detection/mitigation tooling.
- Best-sourced and most balanced threat intelligence summary.

Project Timelines/Deadlines

- Final Demo: 18-19th of September,2026
- Final GitHub Submission: 17th of September, 2026